FINAL Unit4 FRQ - Topic order

1. The Surface class is used to store information about a flat rectangular surface. A partial declaration of the Surface class is shown.

```
public class Surface
{
  /**
   *  Returns the width of the surface, in feet
   */
  public double getWidth()
  { /* implementation not shown */ }

  /**
   *  Returns the length of the surface, in feet
   */
  public double getLength()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The Painter class maintains an array of surfaces that need to be painted.

```
public class Painter
{
  /** The array of surfaces to be painted */
  public Surface[] surfaceArray;

  /**
   *  Returns true if numGallons is enough to
   *  paint all surfaces in surfaceArray and
   *  returns false otherwise
   *  Preconditions: numGallons > 0
   *                 No elements of surfaceArray are null.
   *                 surfaceArray is not empty.
   *  Postcondition: surfaceArray is unchanged.
   */
  public boolean hasEnoughPaint(int numGallons)
  { /*to be implemented */ }
}
```

Write the Painter method hasEnoughPaint. The method should return true if numGallons gallons of paint is enough to paint all surfaces in surfaceArray. The method should return false otherwise. Each gallon of paint is enough to paint an area of 400 square feet.

For example, suppose surfaceArray has the following contents.

FINAL Unit4 FRQ - Topic order

Length	15.0	7.0	12.0	20.0	4.0
Width	5.0	10.5	10.0	20.0	11.5

In this example, the total area of all the surfaces in surfaceArray is 714.5 square feet.

- The call hasEnoughPaint(2) should return true because 2 gallons is enough paint for up to 800 square feet.
- The call hasEnoughPaint(1) should return false because 1 gallon is only enough paint for up to 400 square feet.

Complete method hasEnoughPaint.

```
/**
 * Returns true if numGallons is enough to
 * paint all surfaces in surfaceArray and
 * returns false otherwise
 * Preconditions: numGallons > 0
 *              No elements of surfaceArray are null.
 *              surfaceArray is not empty.
 * Postcondition: surfaceArray is unchanged.
 */
public boolean hasEnoughPaint(int numGallons)
```

2. The ItemInfo class is used to store information about an item at a store. A partial declaration of the ItemInfo class is shown.

```
public class ItemInfo
{
  /**
   * Returns the name of the item
   */
  public String getName()
  { /* implementation not shown */ }

  /**
   * Returns a value greater than 0.0 that represents the cost of a single
   * unit of the item, in dollars
   */
  public double getCost()
  { /* implementation not shown */ }

  /**
   * Returns true if the item is currently available and returns false
   * otherwise
   */
  public boolean isAvailable()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors, and methods
```

FINAL Unit4 FRQ - Topic order

```
    that are not shown. */
}
```

The ItemInventory class maintains an ArrayList named inventory that contains all items at the store. A partial declaration of the ItemInventory class is shown.

```
public class ItemInventory
{
  /** The list of all items at the store */
  private ArrayList<ItemInfo> inventory;

  /**
   * Returns the average cost of the available items whose cost is between
   * lower and upper, inclusive
   * Precondition: lower <= upper
   *          At least one available element of inventory has a cost
   *          between lower and upper, inclusive.
   *          No elements of inventory are null.
   */
  public double averageWithinRange(double lower, double upper)
  { /* to be implemented */}

  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

Write the ItemInventory method averageWithinRange. The method should return the average cost of the available items in inventory whose cost is between the parameters lower and upper, inclusive.

Suppose inventory contains the following seven ItemInfo objects.

Name	"action figure"	"hair brush"	"frying pan"	"dish sponge"	"coffee mug"	"scarf"	"watch"
Cost	20.0	7.99	45.0	2.0	10.0	59.0	45.0
Is Available	true	true	true	false	true	true	false

For the inventory shown, averageWithinRange(10.0, 50.0) should return 25.0, which is equal to the average cost of the available items within the specified range (a 20*actionfigure*, a45 frying pan, and a $10 coffee mug). Although the watch is within the specified range, it is not available.

Complete method averageWithinRange.

```
/**
 * Returns the average cost of the available items whose cost is between
 * lower and upper, inclusive
 * Precondition: lower <= upper
 *          At least one available element of inventory has a cost
 *          between lower and upper, inclusive.
```

FINAL Unit4 FRQ - Topic order

```
 *          No elements of inventory are null.
 */
public double averageWithinRange(double lower, double upper)
```

3. The GameAccount class is used to store information about players of an online game. A partial declaration of the GameAccount class is shown.

```
public class GameAccount
{
  /**
   * Returns the username of the player
   */
  public String getUsername()
  { /* implementation not shown */ }

  /**
   * Returns the age of the player
   */
  public int getAge()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The GamePlatform class maintains an ArrayList of GameAccount objects named allPlayers. A partial declaration of the GamePlatform class is shown.

```
public class GamePlatform
{
  /** An ArrayList of all player accounts */
  public ArrayList<GameAccount> allPlayers;

  /**
   * Returns the average age of players in allPlayers whose age
   * is between 13 and 19, inclusive
   * Preconditions: allPlayers contains at least one player
   *              whose age is between 13 and 19.
   *              allPlayers is not null and contains no null
   *              elements.
   * Postcondition: allPlayers is unchanged.
   */
  public double avgTeenAccounts()
  { /* to be implemented */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

FINAL Unit4 FRQ - Topic order

Write the GamePlatform method avgTeenAccounts. The method should return the average age of the players in allPlayers whose age is between 13 and 19, inclusive.

For example, suppose allPlayers has the following contents:

Username	"star_guy"	"a_knight"	"dino_dan"	"cool_cat"	"happy_hero"	"fox_sox"
Age	23	14	12	15	13	19

For these contents of allPlayers, the method avgTeenAccounts should return 15.25. There are four players whose age is between 13 and 19, inclusive ("a_knight", "cool_cat", "happy_hero", and "fox_sox"). The average age of these four players is $(14 + 15 + 13 + 19)/4$, which is $15.2\overline{5}$.

Complete method avgTeenAccounts.

```
/**
 * Returns the average age of players in allPlayers whose age
 * is between 13 and 19, inclusive
 * Preconditions: allPlayers contains at least one player
 *                whose age is between 13 and 19.
 *                allPlayers is not null and contains no null
 *                elements.
 * Postcondition: allPlayers is unchanged.
 */
public double avgTeenAccounts()
```

4. The Player class is used to store information about a player in a video game. A partial declaration of the Player class is shown.

```
public class Player
{
  /**
   * Returns a unique ID for the player
   */
  public String getID()
  { /* implementation not shown */ }

  /**
   * Returns the player's score in the game,
   * guaranteed to be nonnegative
   */
  public int getScore()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

FINAL Unit4 FRQ - Topic order

The PlayerAnalysis class maintains a list of all players of the game. A partial declaration of the PlayerAnalysis class is shown.

```
public class PlayerAnalysis
{
  /** The list of all players */
  private ArrayList<Player> playerList;

  /**
   * Returns the ID of the player whose score is closest to
   * targetScore
   * Preconditions: playerList is not null.
   *                No elements of playerList are null.
   *                playerList is not empty.
   * Postcondition: playerList is unchanged.
   */
  public String playerWithClosestScore(int targetScore)
  { /* to be implemented */ }

  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

Write the PlayerAnalysis method playerWithClosestScore. The method should return the ID of the player whose score is closest to the parameter targetScore. If multiple players have a score closest to targetScore, then any of their IDs can be returned.

For example, suppose playerList has the following contents.

Player ID	"DJK"	"CJL"	"JOY"	"BEN"	"PTT"	"JAY"
Score	1090	2800	2000	500	3500	4500

For this example, playerWithClosestScore should behave as follows.

- The call playerWithClosestScore(1000) should return "DJK".
- The call playerWithClosestScore(2000) should return "JOY".
- The call playerWithClosestScore(3000) should return "CJL".
- The call playerWithClosestScore(4000) should return either "PTT" or "JAY".
- The call playerWithClosestScore(5000) should return "JAY".

Complete method playerWithClosestScore.

```
/**
 * Returns the ID of the player whose score is closest to
 * targetScore
 * Preconditions: playerList is not null.
```

```
 *            No elements of playerList are null.
 *            playerList is not empty.
 * Postcondition: playerList is unchanged.
 */
public String playerWithClosestScore(int targetScore)
```

5. The Game class is used to store the scores of a home team and an away team in a game. A partial declaration of the Game class is shown.

```
public class Game
{
  /**
   * Returns the home team's score for this game
   */
  public int getHomeScore()
  { /* implementation not shown */ }

  /**
   * Returns the away team's score for this game
   */
  public int getAwayScore()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The Season class maintains an ArrayList of Game objects named allGames. A partial declaration of the Season class is shown.

```
public class Season
{
  /** The list of all games played during the season */
  private ArrayList<Game> allGames;

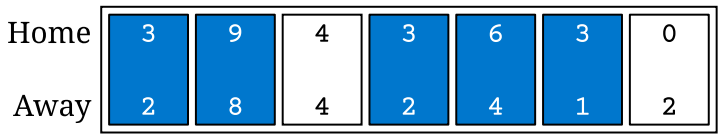
  /**
   * Returns the number of games in the longest streak of
   * consecutive games where the home team scored more than
   * the away team
   * Preconditions: allGames is not empty.
   *            No elements of allGames are null.
   * Postcondition: allGames is unchanged.
   */
  public int getLongestHomeWinStreak()
  { /* to be implemented */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

FINAL Unit4 FRQ - Topic order

Write the Season method getLongestHomeWinStreak. The method should return the number of games that make up the longest winning streak by the home team. A winning streak is a number of consecutive games won by a team. A game is considered a win for the home team if the home team score is greater than the away team score.

For example, suppose allGames contains the following Game objects. The shaded elements represent home team wins.



For these contents of allGames, the method getLongestHomeWinStreak() should return 3 because the longest winning streak for the home team is 3 games.

Complete method getLongestHomeWinStreak.

```
/**
 * Returns the number of games in the longest streak of
 * consecutive games where the home team scored more than
 * the away team
 * Preconditions: allGames is not empty.
 *                No elements of allGames are null.
 * Postcondition: allGames is unchanged.
 */
public int getLongestHomeWinStreak()
```

6. The Shelf class is used to store information about available spare car parts on each shelf in a shop. Each shelf contains a number of axles and a number of wheels, which are two types of car parts. A partial declaration of the Shelf class is shown.

```
public class Shelf
{
  /**
   * Returns the number of axles on the shelf,
   * guaranteed to be 0 or more
   */
  public int getNumAxles()
  { /* implementation not shown */ }

  /**
   * Returns the number of wheels on the shelf
   * guaranteed to be 0 or more
   */
  public int getNumWheels()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

FINAL Unit4 FRQ - Topic order

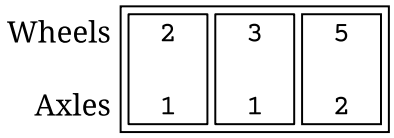
The Shop class maintains an array named shelfArray that represents all the shelves in the shop. A partial declaration of the Shop class is shown.

```
public class Shop
{
  /** An array of all shelves at the shop */
  private Shelf[] shelfArray;

  /**
   * Returns the number of assemblies that can be made from
   * all the parts on all the shelves. Each assembly requires
   * 2 wheels and 1 axle.
   * Preconditions: shelfArray is not null.
   *                no elements of ShelfArray are null.
   * Postcondition: shelfArray is unchanged.
   */
  public int getPossibleAssemblies()
  { /* to be implemented */ }
}
```

An assembly is a car part built using 2 wheels and 1 axle. Write the Shop method getPossibleAssemblies, which should return the maximum number of assemblies that can be made from the available wheels and axles from all shelves.

For example, suppose shelfArray contains the following Shelf objects.



For these contents of shelfArray, the method getPossibleAssemblies() should return 4. There are a total of 10 wheels and 4 axles. Since each assembly requires 2 wheels and 1 axle, there are 4 assemblies possible.

Complete method getPossibleAssemblies.

```
/**
 * Returns the number of assemblies that can be made from
 * all the parts on all the shelves. Each assembly requires
 * 2 wheels and 1 axle.
 * Preconditions: shelfArray is not null.
 *                no elements of ShelfArray are null.
 * Postcondition: shelfArray is unchanged.
 */
public int getPossibleAssemblies()
```

FINAL Unit4 FRQ - Topic order

7. The WordGrid class maintains a two-dimensional array of strings. A partial declaration of the WordGrid class is shown.

```
public class WordGrid
{
  private String[][] grid;

  /**
   * Returns the number of rows in grid that are ordered from
   * shortest string to longest string
   * Preconditions: grid has at least one row and at least two
   *                columns.
   *                No elements in grid are null.
   * Postcondition: grid is unchanged.
   */
  public int countOrderedRows()
  { /* to be implemented */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

When an element of the two-dimensional array grid is accessed, the first index is used to specify the row and the second index is used to specify the column.

Write the WordGrid method countOrderedRows. The method should return the number of rows in grid that are ordered from shortest string to longest string. A row is considered ordered from shortest to longest string if each string element in the row, after the first element, has a length that is greater than or equal to the length of the previous element.

For example, suppose grid has the following contents.

	0	1	2	3
0	"cat"	"crane"	"people"	"hamster"
1	"dog"	"slate"	"bat"	"green"
2	"blue"	"audio"	"adieu"	"snazzy"
3	"yellow"	"shirt"	"hat"	"store"
4	"about"	"media"	"stone"	"canoe"

For these contents of grid, countOrderedRows should return 3 because there are three rows that contain strings that are ordered from shortest string to longest string (rows 0, 2, and 4).

Complete method countOrderedRows.

```
/**
 * Returns the number of rows in grid that are ordered from
```

FINAL Unit4 FRQ - Topic order

```
 * shortest string to longest string
 * Preconditions: grid has at least one row and at least two
 *                columns.
 *                No elements in grid are null.
 * Postcondition: grid is unchanged.
 */
public int countOrderedRows()
```

8. The ElectricVehicle class is used to store information about electric vehicles. A partial declaration of the ElectricVehicle class is shown.

```
public class ElectricVehicle
{
  /**
   * Returns the range of the vehicle, which is the distance, in miles,
   * that the vehicle can travel on a fully charged battery
   */
  public int getRange()
  { /* implementation not shown */ }

  /**
   * Returns the model name of the vehicle
   */
  public String getModelName()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The EVDataSet class maintains a list of electric vehicles. A partial declaration of the EVDataSet class is shown.

```
public class EVDataSet
{
  private ArrayList <ElectricVehicle> vehicleList;

  /**
   * Returns a list of the model names of electric vehicles whose range,
   * in miles, is greater than target, without any duplicate model names
   * Preconditions: vehicleList is not null.
   *                No elements of vehicleList are null.
   * Postcondition: vehicleList is unchanged.
   */
  public ArrayList<String> getHighRangeVehicles(int target)
  { /* to be implemented */}

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

FINAL Unit4 FRQ - Topic order

Write the EVDataSet method getHighRangeVehicles. The method should return an ArrayList containing the model names of all electric vehicles in vehicleList whose range, in miles, is greater than the parameter target.

There may be multiple vehicles in vehicleList with the same model name. If two vehicles have the same model name and both have a range greater than target, the model name should appear only once in the returned list. The model names can appear in any order in the returned ArrayList.

Suppose vehicleList has the following contents.

Range	200	300	250	270	220	190
Model Name	"Eco 6"	"Hallo"	"QTK2"	"Nu EV"	"QTK2"	"Ekstat"

For the vehicleList shown, getHighRangeVehicles(215) should return an ArrayList containing the values "Hallo", "QTK2", and "Nu EV", in any order.

Complete method getHighRangeVehicles.

```
/**
 * Returns a list of the model names of electric vehicles whose range,
 * in miles, is greater than target, without any duplicate model names
 * Preconditions: vehicleList is not null.
 *                No elements of vehicleList are null.
 * Postcondition: vehicleList is unchanged.
 */
public ArrayList<String> getHighRangeVehicles(int target)
```

9. The Passenger class is used to store information about an airplane passenger. A partial declaration of the Passenger class is shown.

```
public class Passenger
{
  /**
   * Returns the loyalty rating of the passenger,
   * guaranteed to be positive
   */
  public int getRating()
  { /* implementation not shown */ }

  /**
   * Returns the passenger's unique ID
   */
  public String getID()
  { /* implementation now shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

FINAL Unit4 FRQ - Topic order

The FlightSeatingChart class represents the assignment of passengers to seats on an airplane flight. The arrangement of seats on an airplane is stored as a two-dimensional array, where each element may or may not be filled by a passenger. A partial declaration of the FlightSeatingChart class is shown.

```
public class FlightSeatingChart
{
  /** 2D array containing all passengers on a flight */
  private Passenger[][] chart;

  /**
   *  Moves the passenger with the highest loyalty rating seated in
   *  a row greater than upgradeRow to chart[upgradeRow][upgradeCol]
   *  Preconditions: upgradeRow and upgradeCol are valid row and
   *                 column indices, respectively, in chart.
   *                 chart[upgradeRow][upgradeCol] is null.
   *                 There is at least one passenger in a row
   *                 greater than upgradeRow.
   */
  public void upgrade(int upgradeRow, int upgradeCol)

  /* There may be instance variables, constructors,
     and methods that are not shown.  */
}
```

When an element of the two-dimensional array chart is accessed, the first index is used to specify the row and the second index is used to specify the column.

Write the FlightSeatingChart method upgrade. The method should find the passenger with the greatest loyalty rating seated in a row greater than upgradeRow and move that passenger to the seat at row upgradeRow and column upgradeCol. The chart element previously occupied by that passenger should be set to null. If there are multiple passengers that meet these criteria, then any of these passengers can be moved.

For example, suppose chart has the following contents. Each empty cell represents null. Each non-empty cell represents a passenger object, where the int is the passenger rating and the String is the passenger ID.

FINAL Unit4 FRQ - Topic order

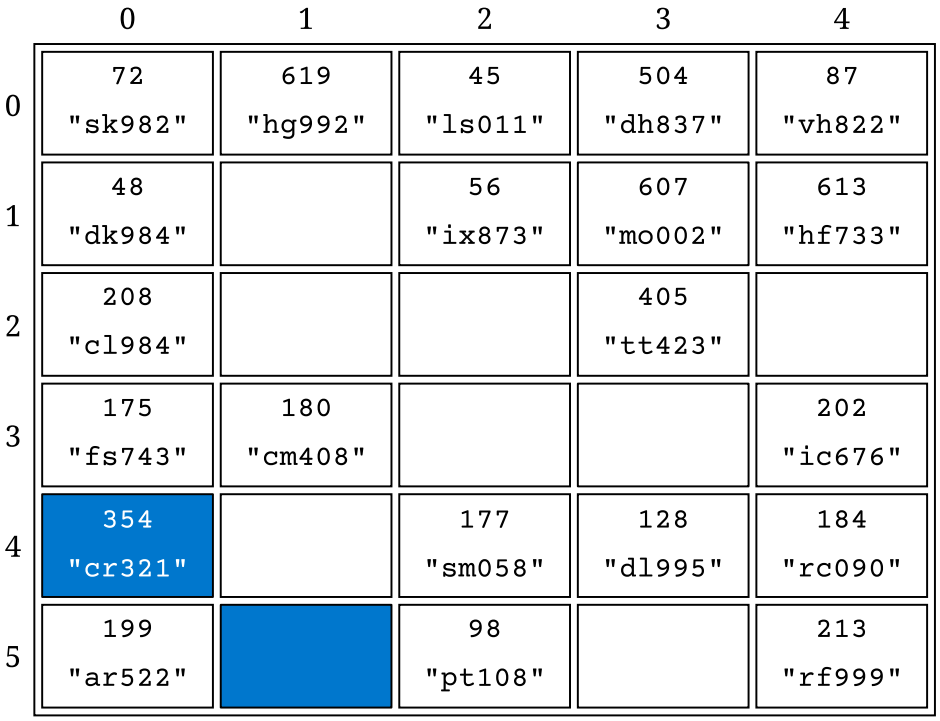
	0	1	2	3	4
0	72 "sk982"	619 "hg992"	45 "ls011"	504 "dh837"	87 "vh822"
1	48 "dk984"		56 "ix873"		613 "hf733"
2	208 "cl984"			405 "tt423"	
3	175 "fs743"	180 "cm408"		607 "mo002"	202 "ic676"
4			177 "sm058"	128 "dl995"	184 "rc090"
5	199 "ar522"	354 "cr321"	98 "pt108"		213 "rf999"

The call upgrade(1, 3) will move the passenger at row 3 column 3 to row 1 column 3, and will set row 3 column 3 to null. The contents of chart after the call are shown in the following figure. The shaded elements are the two that have changed.

	0	1	2	3	4
0	72 "sk982"	619 "hg992"	45 "ls011"	504 "dh837"	87 "vh822"
1	48 "dk984"		56 "ix873"	607 "mo002"	613 "hf733"
2	208 "cl984"			405 "tt423"	
3	175 "fs743"	180 "cm408"			202 "ic676"
4			177 "sm058"	128 "dl995"	184 "rc090"
5	199 "ar522"	354 "cr321"	98 "pt108"		213 "rf999"

The call upgrade(4, 0) will move the passenger at row 5 column 1 to row 4 column 0, and will set row 5 column 1 to null. The contents of chart after the call are shown in the following figure. The shaded elements are the two that have changed.

FINAL Unit4 FRQ - Topic order



	0	1	2	3	4
0	72 "sk982"	619 "hg992"	45 "ls011"	504 "dh837"	87 "vh822"
1	48 "dk984"		56 "ix873"	607 "mo002"	613 "hf733"
2	208 "cl984"			405 "tt423"	
3	175 "fs743"	180 "cm408"			202 "ic676"
4	354 "cr321"		177 "sm058"	128 "dl995"	184 "rc090"
5	199 "ar522"		98 "pt108"		213 "rf999"

Complete method upgrade.

```
/**
 * Moves the passenger with the highest loyalty rating seated in
 * a row greater than upgradeRow to chart[upgradeRow][upgradeCol].
 * Preconditions: upgradeRow and upgradeCol are valid row and
 *                column indices, respectively, in chart.
 *                chart[upgradeRow][upgradeCol] is null.
 *                There is at least one passenger in a row
 *                greater than upgradeRow.
 */
public void upgrade(int upgradeRow, int upgradeCol)
```

10. The GameGrid class is used to move a game piece diagonally on a two-dimensional grid. A partial declaration of the class is shown.

```
public class GameGrid
{
  /**
   * A grid that a game piece moves on. Each element indicates the
   * direction the game piece should move next (either "NE", "NW",
   * "SE", or "SW").
   */
  private String[][] grid;

  /**
   * Computes and returns the number of moves that are taken
```

FINAL Unit4 FRQ - Topic order

```
   * by the game piece when following a path through the grid
   * Preconditions: startRow is a valid row index.
   *                startCol is a valid column index.
   *                There exists a path that moves the game piece
   *                from the starting position to past the
   *                boundaries of the grid.
   * Postcondition: grid is unchanged.
   */
  public int navigate(int startRow, int startCol)
  { /* to be implemented */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

When an element of the two-dimensional array grid is accessed, the first index is used to specify the row and the second index is used to specify the column.

The navigate method has two parameters that represent the starting row and column of the game piece. Each element of the two-dimensional array grid is equal to one of four String values: "NE", "NW", "SE", or "SW". These values indicate the direction the game piece should be moved, as described in the following table.

Direction	Effect
"NE"	Decreases the current row by one and increases the current column by one
"NW"	Decreases the current row by one and decreases the current column by one
"SE"	Increases the current row by one and increases the current column by one
"SW"	Increases the current row by one and decreases the current column by one

The game piece continues to move according to the values in grid until the current row or current column is outside the boundaries of grid. The method should return the number of valid grid elements accessed.

For example, suppose grid has the following contents.

FINAL Unit4 FRQ - Topic order

	0	1	2	3	4	5	6
0	"SE"	"SE"	"NE"	"NW"	"NW"	"SW"	"SE"
1	"NE"	"SE"	"NE"	"SE"	"NE"	"SE"	"NE"
2	"NW"	"NW"	"NE"	"NW"	"SE"	"SW"	"SE"
3	"SE"	"SW"	"SE"	"SW"	"NE"	"SW"	"NW"
4	"SW"	"NE"	"NW"	"NE"	"SE"	"NE"	"SW"

The call navigate(0, 0) will cause the game piece to move along the path indicated by the shaded elements in the following figure. The method should return 7 because there are seven elements accessed.

	0	1	2	3	4	5	6
0	**"SE"**	"SE"	"NE"	"NW"	"NW"	"SW"	"SE"
1	"NE"	**"SE"**	"NE"	**"SE"**	"NE"	"SE"	"NE"
2	"NW"	"NW"	**"NE"**	"NW"	**"SE"**	"SW"	"SE"
3	"SE"	"SW"	"SE"	"SW"	"NE"	**"SW"**	"NW"
4	"SW"	"NE"	"NW"	"NE"	**"SE"**	"NE"	"SW"

The call navigate(2, 1) will cause the game piece to move along the path indicated by the shaded elements in the following figure. The method should return 5 because there are five elements accessed.

FINAL Unit4 FRQ - Topic order

	0	1	2	3	4	5	6
0	"SE"	"SE"	"NE"	"NW"	"NW"	"SW"	"SE"
1	"NE"	"SE"	"NE"	"SE"	"NE"	"SE"	"NE"
2	"NW"	"NW"	"NE"	"NW"	"SE"	"SW"	"SE"
3	"SE"	"SW"	"SE"	"SW"	"NE"	"SW"	"NW"
4	"SW"	"NE"	"NW"	"NE"	"SE"	"NE"	"SW"

Complete method navigate.

```
/**
 * Computes and returns the number of moves that are taken
 * by the game piece when following a path through the grid
 * Preconditions: startRow is a valid row index.
 *                startCol is a valid column index.
 *                There exists a path that moves the game piece
 *                from the starting position to past the
 *                boundaries of the grid.
 * Postcondition: grid is unchanged.
 */
public int navigate(int startRow, int startCol)
```

11. **SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.**

Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves the manipulation and analysis of a list of words. The following `WordChecker` class contains an `ArrayList<String>` to be analyzed and methods that are used to perform the analysis. You will write two methods of the `WordChecker` class.

```
public class WordChecker
{
/** Initialized in the constructor and contains no null elements */
private ArrayList<String> wordList;
/**
 * Returns true if each element of wordList (except the first) contains the
previous
```

FINAL Unit4 FRQ - Topic order

```
 * element as a substring and returns false otherwise, as described in part (a)
 * Precondition: wordList contains at least two elements.
 * Postcondition: wordList is unchanged.
 */
public boolean isWordChain()
{ /* to be implemented in part (a) */ }
/**
 * Returns an ArrayList<String> based on strings from wordList that start
 * with target, as described in part (b). Each element of the returned ArrayList has had
 * the initial occurrence of target removed.
 * Postconditions: wordList is unchanged.
 *        Items appear in the returned list in the same order as they appear in wordList.
 */
public ArrayList<String> createList(String target)
{ /* to be implemented in part (b) */ }
// There may be instance variables, constructors, and methods that are not shown.
}
```

Write the `isWordChain` method, which determines whether each element of `wordList` (except the first) contains the previous element as a substring. The following table shows two sample `isWordChain` method calls.

wordList	isWordChain Return Value	Explanation
`["an", "band", "band", "abandon"]`	`true`	Each element contains the previous element as a substring.
`["to", "too", "stool", "tools"]`	`false`	`"tools"` does not contain the substring `"stool"`.

Complete the `isWordChain` method.

```
/**
* Returns true if each element of wordList (except the first) contains the previous
* element as a substring and returns false otherwise, as described in part (a)
* Precondition: wordList contains at least two elements.
* Postcondition: wordList is unchanged.
*/
public boolean isWordChain()
```

Write the `createList` method, which creates and returns an `ArrayList<String>`. The method identifies strings in `wordList` that start with `target` and returns a new `ArrayList` containing each identified string without the starting occurrence of `target`. Elements must appear in the returned list in the same order as they appear in `wordList`.

Consider an example where `wordList` contains the following strings.

```
["catch", "bobcat", "catchacat", "cat", "at"]
```

The following table shows the `ArrayList` returned by some calls to `createList`. In all cases, `wordList` is unchanged.

Method Call	`ArrayList` Returned by `createList`	Explanation
`createList("cat")`	`["ch", "chacat", ""]`	Only `"catch"`, `"catchacat"`, and `"cat"` begin with `"cat"`.
`createList("catch")`	`["", "acat"]`	Only `"catch"` and `"catchacat"` begin with `"catch"`.
`createList("dog")`	`[]`	None of the words in `wordList` begin with `"dog"`.

Complete the `createList` method.

```
/**
* Returns an ArrayList<String> based on strings from wordList that start
* with target, as described in part (b). Each element of the returned ArrayList has had
* the initial occurrence of target removed.
* Postconditions: wordList is unchanged.
* Items appear in the returned list in the same order as they appear in wordList.
*/
public ArrayList<String> createList(String target)
```

Class information for this question

```
public class WordChecker
private ArrayList<String> wordList
public boolean isWordChain()
public ArrayList<String> createList(String target)
```

12. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves pieces of candy in a box. The `Candy` class represents a single piece of candy.

```
public class Candy
{
 /** Returns a String representing the flavor of this piece of candy */
```

FINAL Unit4 FRQ - Topic order

```
    public String getFlavor()
    { /* implementation not shown */ }


    // There may be instance variables, constructors, and methods that are not
    shown.
}
```

The `BoxOfCandy` class represents a candy box where the candy is arranged in a rectangular grid. The instance variable of the class, `box`, is a rectangular two-dimensional array of `Candy` objects. A location in the candy box may contain a piece of candy or may be empty. A piece of candy is represented by a `Candy` object. An empty location is represented by `null`. You will write two methods of the `BoxOfCandy` class.

```
public class BoxOfCandy
{
    /** box contains at least one row and is initialized in the constructor. */
    private Candy[][] box;


    /**
     * Moves one piece of candy in column col, if necessary and possible, so that
     the box
     * element in row 0 of column col contains a piece of candy, as described in
     part (a).
     * Returns false if there is no piece of candy in column col and returns true
     otherwise.
     * Precondition: col is a valid column index in box.
     */
    public boolean moveCandyToFirstRow(int col)
    { /* to be implemented in part (a) */ }


    /**
     * Removes from box and returns a piece of candy with flavor specified by the
     parameter, or
     * returns null if no such piece is found, as described in part (b)
     */
    public Candy removeNextByFlavor(String flavor)
    { /* to be implemented in part (b) */ }

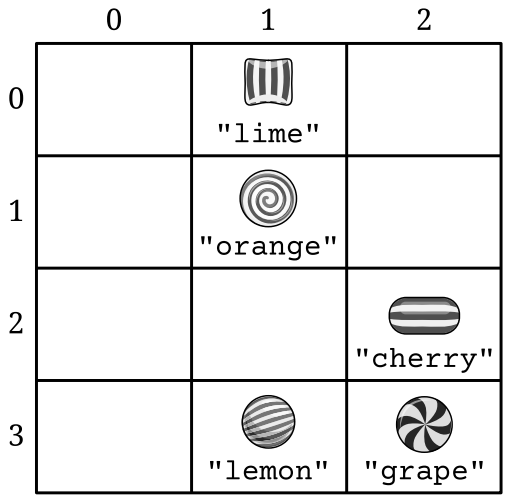

    // There may be instance variables, constructors, and methods that are not
    shown.
}
```

FINAL Unit4 FRQ - Topic order

Write the `moveCandyToFirstRow` method, which attempts to ensure that the `box` element at row `0` and column `col` contains a piece of candy, using the following steps.

If the element at row `0` and column `col` already contains a piece of candy, then `box` is unchanged and the method returns `true`. If the element at row `0` and column `col` does not contain a piece of candy, then the method searches the remaining rows of column `col` for a piece of candy. If a piece of candy can be found in column `col`, it is moved to row `0`, its previous location is set to `null`, and the method returns `true`; otherwise, the method returns `false`.

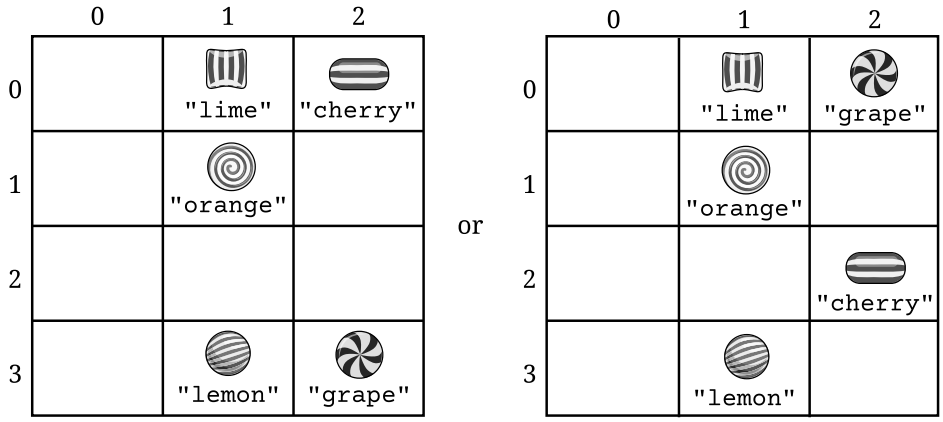
In the following example, the grid represents the contents of `box`. An empty square in the grid is `null` in `box`. A non-empty square in the grid represents a `box` element that contains a `Candy` object. The string in the square of the grid indicates the flavor of the piece of candy.



The method call `moveCandyToFirstRow(0)` returns `false` because the `box` element at row `0` and column `0` does not contain a piece of candy and there are no pieces of candy in column `0` that can be moved to row `0`. The contents of `box` are unchanged.

The method call `moveCandyToFirstRow(1)` returns `true` because the `box` element at row `0` and column `1` already contains a piece of candy. The contents of `box` are unchanged.

The method call `moveCandyToFirstRow(2)` moves one of the two pieces of candy in column `2` to row `0` of column `2`, sets the previous location of the piece of candy that was moved to `null`, and returns `true`. The new contents of `box` could be either of the following.



Complete the `moveCandyToFirstRow` method.

```
/**
* Moves one piece of candy in column col, if necessary and possible, so that
the box
* element in row 0 of column col contains a piece of candy, as described in
part (a).
```

FINAL Unit4 FRQ - Topic order

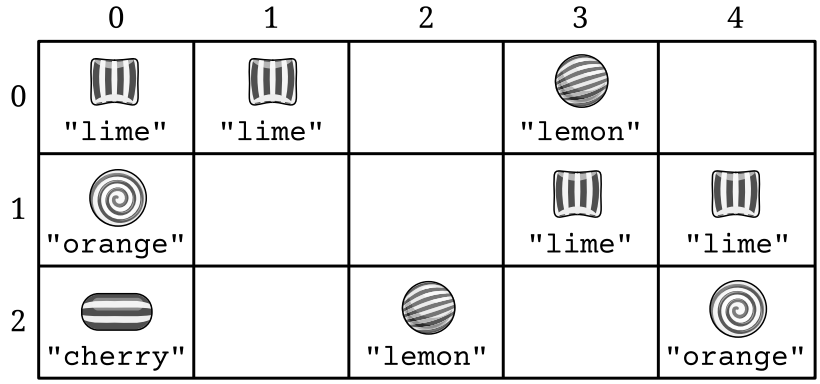
```
* Returns false if there is no piece of candy in column col and returns true
otherwise.
* Precondition: col is a valid column index in box.
*/
public boolean moveCandyToFirstRow(int col)
```

Write the `removeNextByFlavor` method, which attempts to remove and return one piece of candy from the box. The piece of candy to be removed is the first piece of candy with a flavor equal to the parameter `flavor` that is encountered while traversing the candy box in the following order: the last row of the box is traversed from left to right, then the next-to-last row of the box is traversed from left to right, etc., until either a piece of candy with the desired flavor is found or until the entire candy box has been searched.

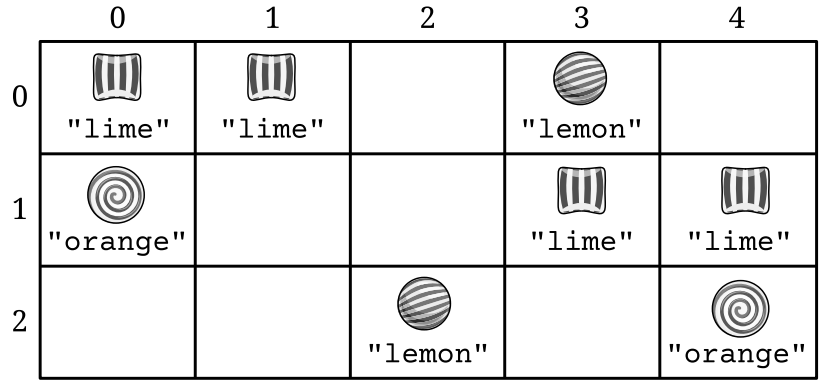
If the `removeNextByFlavor` method finds a `Candy` object with the desired flavor, the corresponding `box` element is assigned `null`, all other `box` elements are unchanged, and the removed `Candy` object is returned. Otherwise, `box` is unchanged and the method returns `null`.

The following examples show three consecutive calls to the `removeNextByFlavor` method. The traversal of the candy box always begins in the last row and first column of the box.

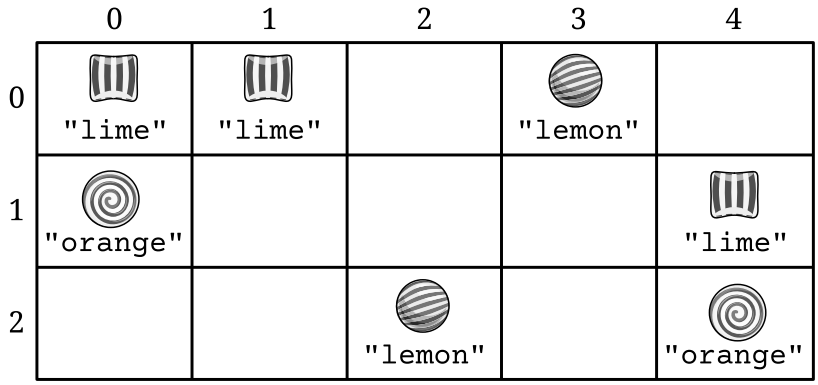
The following grid shows the contents of `box` before any of the `removeNextByFlavor` method calls.



The method call `removeNextByFlavor("cherry")` removes and returns the `Candy` object located in row `2` and column `0`. The following grid shows the updated contents of `box`.



The method call `removeNextByFlavor("lime")` removes and returns the `Candy` object located in row `1` and column `3`. The following grid shows the updated contents of `box`.



The method call `removeNextByFlavor("grape")` returns `null` because no grape-flavored candy is found. The contents of `box` are unchanged.

Complete the `removeNextByFlavor` method.

```
/**
 * Removes from box and returns a piece of candy with flavor specified by the
parameter, or
 * returns null if no such piece is found, as described in part (b)
 */
public Candy removeNextByFlavor(String flavor)
```

13. Directions: SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Notes:

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

1. This question involves analyzing the difficulty levels of trivia questions used in a trivia game. Trivia questions are represented by the following `TriviaQuestion` class.

```
public class TriviaQuestion
{
 /** Returns the difficulty level of the trivia question, which is a value
 between 0 and
  *  100, inclusive
  */
 public int getDifficultyLevel()
 {  /* implementation not shown */  }
```

FINAL Unit4 FRQ - Topic order

```
// There may be instance variables, constructors, and methods that are not
shown.
}
```

An array of the trivia questions that make up a trivia game is stored in a `TriviaGame` object. A partial declaration of the `TriviaGame` class is shown below. You will write two methods of the `TriviaGame` class.

```
public class TriviaGame
{
/** An array of all trivia questions in the game
  *   Guaranteed to have at least three entries and to contain only non-null
entries
 */
private TriviaQuestion[] allQuestions;

/** Returns the greatest difficulty level that appears in any trivia question
in the game */
public int getMostDifficult()
{  /* implementation not shown */  }

/** Returns the least difficulty level that appears in any trivia question in
the game */
public int getLeastDifficult()
{  /* implementation not shown */  }

/** Returns a String representing the overall rating of the trivia game as
  *  "Beginner", "Intermediate", or "Expert", as described in part (a)
 */
public String getRating()
{  /* to be implemented in part (a) */  }

/** Returns true if the trivia questions in allQuestions appear in order from
least
  *   difficulty to greatest difficulty and returns false otherwise, as
described in part (b)
 */
public boolean isDifficultyIncreasing()
{  /* to be implemented in part (b) */  }

// There may be instance variables, constructors, and methods that are not
shown.
}
```

(a) Write the `TriviaGame` method `getRating`, which returns a `String` representing the difficulty level of the trivia game based on the *adjusted average* of the difficulty levels of the trivia questions in the game.

FINAL Unit4 FRQ - Topic order

The *adjusted average* is defined as the average of the difficulty levels with the least value and greatest value ignored. If there are multiple questions with the greatest difficulty level, only one is ignored. If there are multiple questions with the least difficulty level, only one is ignored. A trivia game consists of at least three trivia questions.

Based on the adjusted average, the `getRating` method should return the following values.

- `"Expert"` if the adjusted average is at least `70.0`
- `"Intermediate"` if the adjusted average is at least `50.0` but less than `70.0`
- `"Beginner"` if the adjusted average is less than `50.0`

For example, assume that `game1` has been declared as a `TriviaGame` object and that `q1`, `q2`, `q3`, and `q4` are properly declared and initialized `TriviaQuestion` objects. The following is an example of the contents of `allQuestions` in `game1`, showing each `TriviaQuestion` object and its difficulty level.

0	1	2	3
q1 Difficulty: 50	q2 Difficulty: 90	q3 Difficulty: 85	q4 Difficulty: 90

In computing the adjusted average for the `game1` object, the least value (`50`) and greatest value (`90`) are ignored. The adjusted average (`87.5`) is computed as the average of the remaining two values, `85` and `90`.

The method call `game1.getRating()` should return `"Expert"`.

Two helper methods, `getMostDifficult` and `getLeastDifficult`, are provided in the `TriviaGame` class. You must use `getMostDifficult` and `getLeastDifficult` appropriately to receive full credit.

Complete method `getRating`.

```
/** Returns a String representing the overall rating of the trivia game as
   *  "Beginner", "Intermediate", or "Expert", as described in part (a)
 */
public String getRating()
```

(b) Write the `TriviaGame` method `isDifficultyIncreasing`, which returns `true` if the difficulty level of each question in `allQuestions` is less than or equal to the question that follows it, if any, and returns `false` otherwise.

For example, assume that `game1` and `game2` have been declared as `TriviaGame` objects and that `q1`, `q2`, `q3`, `q4`, `q5`, and `q6` are properly declared and initialized `TriviaQuestion` objects. The following example shows the contents of `allQuestions` for `game1` and `game2`. The `TriviaQuestion` object and its difficulty level are provided for each `allQuestions` element.

The array `allQuestions` for `game1` object:

0	1	2	3	4
q1 Difficulty: 50	q2 Difficulty: 60	q3 Difficulty: 60	q4 Difficulty: 70	q5 Difficulty: 80

The array `allQuestions` for `game2` object:

0	1	2	3
q1 Difficulty: 50	q4 Difficulty: 70	q3 Difficulty: 60	q6 Difficulty: 90

The method call `game1.isDifficultyIncreasing()` should return `true`, and the method call `game2.isDifficultyIncreasing()` should return `false`.

Complete method `isDifficultyIncreasing`.

```
/** Returns true if the trivia questions in allQuestions appear in order from
least
  *   difficulty to greatest difficulty and returns false otherwise, as
described in part (b)
 */
public boolean isDifficultyIncreasing()
```

(c) A programmer would like to add a method called `gameLength` to inform players of how long it is expected to take, in minutes, to play a trivia game. The expected length of a game is based on the expected time to answer each question in the game.

Write a description of how you would change the `TriviaQuestion` and `TriviaGame` classes in order to support this modification.

Make sure to include the following in your response.

- Write the method header for the `gameLength` method.
- Identify any new or modified instance variables, constructors, or methods aside from the `gameLength` method. **Do not write the program code for this change.**
- Describe, for each new or revised variable, constructor, or method, how it would change or be implemented, including visibility and type. You do not need to describe the implementation of the `gameLength` method. **Do not write the program code for this change.**

14. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Assume that the classes listed in the Java Quick Reference have been imported where appropriate.

Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.

In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

Employees at a store are paid daily wages according to the following rules.

Each employee is paid the same fixed amount per day.

Each employee is paid an additional amount for each item they sold on that day.

Daily Bonus:	If the number of items sold that day by an employee is greater than a computed threshold, then the employee also receives a bonus equal to 10 percent of the employee's daily wages.

You will write two methods in the `Payroll` class below.

```
public class Payroll
{
private int[] itemsSold; // number of items sold by each employee
private double[] wages; // wages to be computed in part (b)
```

FINAL Unit4 FRQ - Topic order

```
/** Returns the bonus threshold as described in part (a).
 */
public double computeBonusThreshold()
{
    /* To be implemented in part (a) */
}

/** Computes employee wages as described in part (b)
 * and stores them in wages.
 * The parameter fixedWage represents the fixed amount each employee
 * is paid per day.
 * The parameter perItemWage represents the amount each employee
 * is paid per item sold.
 */
public void computeWages(double fixedWage, double perItemWage)
{
    /* To be implemented in part (b) */
}

// Other instance variables, constructors, and methods not shown.
}
```

The bonus threshold is calculated based on the number of items sold by all employees on a given day. The employee with the greatest number of sales and the employee with the least number of sales on that day are ignored in the calculation. The average number of items sold by the remaining employees on that day is computed, and this value is used as the bonus threshold.

For a given day, the number of items sold by each employee is stored in the array `itemsSold`. The example below shows the contents of `itemsSold` for a day in which there were ten employees. Each array index represents an individual employee. For example, `itemsSold[3]` represents the number of items sold by employee 3.

	0	1	2	3	4	5	6	7	8	9
itemsSold	48	50	37	62	38	70	55	37	64	60

Based on the information in the table, the bonus threshold is calculated as follows.

$\frac{(48+50+37+62+38+70+55+37+64+60)-37-70}{8} = 51.75$

(a) Complete the method `computeBonusThreshold` below, which is intended to return the bonus threshold based on the contents of the `itemsSold` array. Assume that `itemsSold` has been filled appropriately, and that the array contains at least three employees.

```
/** Returns the bonus threshold as described in part (a).
*/
public double computeBonusThreshold()
```

FINAL Unit4 FRQ - Topic order

The `computeWages` method is intended to calculate the wages for each employee and to assign them to the appropriate element of the array `wages`. For example, `wages[3]` should be assigned the wages for employee 3. An employee's wages consist of their daily wages plus a possible bonus and are calculated as follows.

Each employee's wages are equal to the fixed wage plus the number of items sold times the amount paid per item sold. If the employee sold more items than the bonus threshold, the employee also receives a 10 percent bonus added to their wages.

As described in part (a), `computeBonusThreshold()` returns `51.75` for the example array below.

	0	1	2	3	4	5	6	7	8	9
itemsSold	48	50	37	62	38	70	55	37	64	60

Suppose that `fixedWage` is `10.0` and `perItemWage` is `1.5`.

Employee 0 did not sell more items than the bonus threshold, so employee 0's wages are equal to `10.0 + 1.5 * 48`, which evaluates to `82.0`. This value will be assigned to `wages[0]`.
Employee 9 sold more items than the bonus threshold, so employee 9 receives a 10 percent bonus. Employee 9's wages are equal to `(10.0 + 1.5 * 60) * 1.1`, or `110.0`. This value will be assigned to `wages[9]`.

(b) Write the method `computeWages`. Assume that `itemsSold` has been filled appropriately, and there are at least three employees in the array. Assume also that the `wages` array and the `itemsSold` array have the same length. Your solution must call `computeBonusThreshold` appropriately to receive full credit.

```
/** Computes employee wages as described in part (b)
* and stores them in wages.
* The parameter fixedWage represents the fixed amount each employee
* is paid per day.
* The parameter perItemWage represents the amount each employee
* is paid per item sold.
*/
public void computeWages(double fixedWage, double perItemWage)
```

15. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves the analysis of weather data. The following `WeatherData` class has an instance variable, `temperatures`, which contains the daily high temperatures recorded on consecutive days at a particular location. The class also contains methods used to analyze that data. You will write two methods of the `WeatherData` class.

```
public class WeatherData
{
 /** Guaranteed not to be null and to contain only non-null entries */
```

FINAL Unit4 FRQ - Topic order

```
private ArrayList<Double> temperatures;


/**
 * Cleans the data by removing from temperatures all values that are less than
 * lower and all values that are greater than upper, as described in part (a)
 */
public void cleanData(double lower, double upper)
{ /* to be implemented in part (a) */ }


/**
 * Returns the length of the longest heat wave found in temperatures, as
described in
 * part (b)
 * Precondition: There is at least one heat wave in temperatures based on
threshold.
 */
public int longestHeatWave(double threshold)
{ /* to be implemented in part (b) */ }


// There may be instance variables, constructors, and methods that are not
shown.
}
```

Write the `cleanData` method, which modifies the `temperatures` instance variable by removing all values that are less than the `lower` parameter and all values that are greater than the `upper` parameter. The order of the remaining values in `temperatures` must be maintained.

For example, consider a `WeatherData` object for which `temperatures` contains the following.

99.1	142.0	85.0	85.1	84.6	94.3	124.9	98.0	101.0	102.5

The three shaded values shown would be removed by the method call `cleanData(85.0, 120.0)`.

99.1	142.0	85.0	85.1	84.6	94.3	124.9	98.0	101.0	102.5

The following shows the contents of `temperatures` after the three shaded values are removed as a result of the method call `cleanData(85.0, 120.0)`.

99.1	85.0	85.1	94.3	98.0	101.0	102.5

Complete method `cleanData`.

```
/**
* Cleans the data by removing from temperatures all values that are less than
* lower and all values that are greater than upper, as described in part (a)
*/
```

FINAL Unit4 FRQ - Topic order

```
public void cleanData(double lower, double upper)
```

Write the `longestHeatWave` method, which returns the length of the longest heat wave found in the `temperatures` instance variable. A heat wave is a sequence of two or more consecutive days with a daily high temperature greater than the parameter `threshold`. The `temperatures` instance variable is guaranteed to contain at least one heat wave based on the `threshold` parameter.

For example, consider the following contents of `temperatures`.

100.5	98.5	102.0	103.9	87.5	105.2	90.3	94.8	109.1	102.1	107.4	93.2

In the following sample contents of `temperatures`, all heat waves based on the `threshold` temperature of `100.5` are shaded. The method call `longestHeatWave(100.5)` would return `3`, which is the length of the longest heat wave.

100.5	98.5	102.0	103.9	87.5	105.2	90.3	94.8	109.1	102.1	107.4	93.2

In the following sample contents of `temperatures`, all heat waves based on the `threshold` temperature of `95.2` are shaded. The method call `longestHeatWave(95.2)` would return `4`, which is the length of the longest heat wave.

100.5	98.5	102.0	103.9	87.5	105.2	90.3	94.8	109.1	102.1	107.4	93.2

Complete method `longestHeatWave`.

```
/**
* Returns the length of the longest heat wave found in temperatures, as
described in
* part (b)
* Precondition: There is at least one heat wave in temperatures based on
threshold.
*/
public int longestHeatWave(double threshold)
```

16. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

Many encoded strings contain *delimiters*. A delimiter is a non-empty string that acts as a boundary between different parts of a larger string. The delimiters involved in this question occur in pairs that must be *balanced*, with each pair having an open delimiter and a close delimiter. There will be only one type of delimiter for each string. The following are examples of delimiters.

Example 1

Expressions in mathematics use open parentheses `"("` and close parentheses `")"` as delimiters. For each open parenthesis, there must be a matching close parenthesis.

`(x + y) * 5` is a valid mathematical expression.

FINAL Unit4 FRQ - Topic order

`(x + (y)`	is NOT a valid mathematical expression because there are more open delimiters than close delimiters.

Example 2
HTML uses `<B>` and `</B>` as delimiters. For each open delimiter `<B>`, there must be a matching close delimiter `</B>`.

`<B> Make this text bold </B>`	is valid HTML.
`<B> Make this text bold </UB>`	is NOT valid HTML because there is one open delimiter and no matching close delimiter.

In this question, you will write two methods in the following `Delimiters` class.

```
public class Delimiters
{

/** The open and close delimiters. */
private String openDel;
private String closeDel;

/** Constructs a Delimiters object where open is the open delimiter and close
is the
 *  close delimiter.
 *  Precondition: open and close are non-empty strings.
 */
public Delimiters(String open, String close)
{
   openDel = open;
   closeDel = close;
}

/** Returns an ArrayList of delimiters from the array tokens, as described in
part (a). */
public ArrayList<String> getDelimitersList(String[] tokens)
{  /* to be implemented in part (a) */  }

/** Returns true if the delimiters are balanced and false otherwise, as
described in part (b).
 *  Precondition: delimiters contains only valid open and close delimiters.
 */
public boolean isBalanced(ArrayList<String> delimiters)
{  /* to be implemented in part (b) */  }

// There may be instance variables, constructors, and methods that are not
shown.
```

FINAL Unit4 FRQ - Topic order

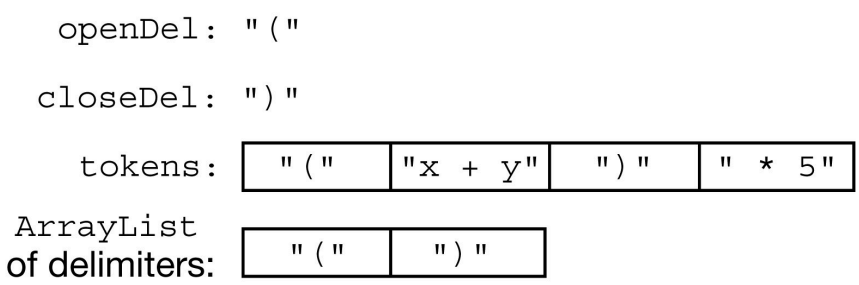
```
}
```

(a)

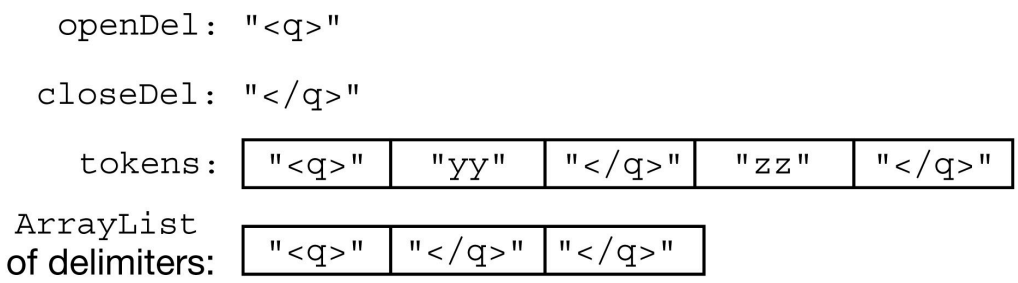
A string containing text and possibly delimiters has been split into *tokens* and stored in `String[] tokens`. Each token is either an open delimiter, a close delimiter, or a substring that is not a delimiter. You will write the method `getDelimitersList`, which returns an `ArrayList` containing all the open and close delimiters found in `tokens` in their original order.

The following examples show the contents of an `ArrayList` returned by `getDelimitersList` for different open and close delimiters and different `tokens` arrays.

Example 1



Example 2



Class information for this question
`public class Delimiters`
`private String openDel`
`private String closeDel`
`public Delimiters(String open, String close)`
`public ArrayList<String> getDelimitersList(String[] tokens)`
`public boolean isBalanced(ArrayList<String> delimiters)`

Complete method `getDelimitersList` below.

```
/** Returns an ArrayList of delimiters from the array tokens, as described in
part (a). */
public ArrayList<String> getDelimitersList(String[] tokens)
```

(b)

Write the method `isBalanced`, which returns `true` when the delimiters are balanced and returns `false` otherwise. The delimiters are balanced when both of the following conditions are satisfied; otherwise, they are not balanced.

FINAL Unit4 FRQ - Topic order

1. When traversing the `ArrayList` from the first element to the last element, there is no point at which there are more close delimiters than open delimiters at or before that point.
2. The total number of open delimiters is equal to the total number of close delimiters.

Consider a `Delimiters` object for which `openDel` is "^{" and `closeDel` is "}". The examples below show different `ArrayList` objects that could be returned by calls to `getDelimitersList` and the value that would be returned by a call to `isBalanced`.

Example 1
The following example shows an `ArrayList` for which `isBalanced` returns `true`. As tokens are examined from first to last, the number of open delimiters is always greater than or equal to the number of close delimiters. After examining all tokens, there are an equal number of open and close delimiters.

"^{"	"^{"	"}"	"^{"	"}"	"}"

Example 2
The following example shows an `ArrayList` for which `isBalanced` returns `false`.

"^{"	"}"	"</sup>"	"<sup>"

When starting from the left, at this point, condition 1 is violated.

Example 3
The following example shows an `ArrayList` for which `isBalanced` returns `false`.

"</sup>"

At this point, condition 1 is violated.

Example 4
The following example shows an `ArrayList` for which `isBalanced` returns `false` because the second condition is violated. After examining all tokens, there are not an equal number of open and close delimiters.

"<sup>"	"^{"	"}"

Class information for this question

```
public class Delimiters
private String openDel
private String closeDel
public Delimiters(String open, String close)
public ArrayList<String> getDelimitersList(String[] tokens)
public boolean isBalanced(ArrayList<String> delimiters)
```

Complete method `isBalanced` below.

```
/** Returns true if the delimiters are balanced and false otherwise, as
described in part (b).
*  Precondition: delimiters contains only valid open and close delimiters.
*/
```

```
public boolean isBalanced(ArrayList<String> delimiters)
```

17. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves the creation of user names for an online system. A user name is created based on a user's first and last names. A new user name cannot be a duplicate of a user name already assigned. You will write the constructor and one method of the `UserName` class. A partial declaration of the `UserName` class is shown below.

```
public class UserName
{
// The list of possible user names, based on a user's first and last names and
initialized by the constructor.
private ArrayList<String> possibleNames;

/** Constructs a UserName object as described in part (a).
 * Precondition: firstName and lastName have length greater than 0
 * and contain only uppercase and lowercase letters.
 */
public UserName(String firstName, String lastName)
{ /* to be implemented in part (a) */ }

/** Returns true if arr contains name, and false otherwise. */
public boolean isUsed(String name, String[] arr)
{ /* implementation not shown */ }

/** Removes strings from possibleNames that are found in usedNames as
described in part (b).
 */
public void setAvailableUserNames(String[] usedNames)
{ /* to be implemented in part (b) */ }
}
```

(a) Write the constructor for the `UserName` class. The constructor initializes and fills `possibleNames` with possible user names based on the `firstName` and `lastName` parameters. The possible user names are obtained by concatenating `lastName` with different substrings of `firstName`. The substrings begin with the first character of `firstName` and the lengths of the substrings take on all values from 1 to the length of `firstName`.

The following example shows the contents of `possibleNames` after a `UserName` object has been instantiated.

Example

```
UserName person = new UserName("john", "smith");
```

FINAL Unit4 FRQ - Topic order

After the code segment has been executed, the `possibleNames` instance variable of `person` will contain the following `String` objects in some order.

`"smithj"`, `"smithjo"`, `"smithjoh"`, `"smithjohn"`

Write the `UserName` constructor below.

```
/** Constructs a UserName object as described in part (a).
* Precondition: firstName and lastName have length greater than 0
* and contain only uppercase and lowercase letters.
*/
public UserName(String firstName, String lastName)
```

(b) Write the `UserName` method `setAvailableUserNames`. The method removes from `possibleNames` all names that are found in `usedNames`. These represent user names that have already been assigned in the online system and are therefore unavailable.

A helper method, `isUsed`, has been provided. The `isUsed` method searches for `name` in `arr`. The method returns `true` if an exact match is found and returns `false` otherwise.

The example below shows the intended behavior of the `setAvailableUserNames` method.

Statement	**Strings in `possibleNames` after statement execution**
`String[] used = {"harta", "hartm", "harty"};`	
`UserName person2 = new UserName("mary", "hart");`	`"hartm"`, `"hartma"`, `"hartmar"`, `"hartmary"`
`person2.setAvailableUserNames(used);`	`"hartma"`, `"hartmar"`, `"hartmary"`

Assume that the constructor works as specified, regardless of what you wrote in part (a). You must use `isUsed` appropriately to receive full credit.

Complete the `setAvailableUserNames` method below.

```
/** Removes strings from possibleNames that are found in usedNames as
described in part (b).
*/
public void setAvailableUserNames(String[] usedNames)
```

18. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves a path through a two-dimensional (2D) array of integers, where the path is based on the values of elements in the array. When an element of the 2D array is accessed, the first index is used to specify the row and the second index is used to specify the column. The following `Location` class represents a row and column position in the 2D array.

```
public class Location
```

FINAL Unit4 FRQ - Topic order

```
{
private int theRow;
private int theCol;
public Location(int r, int c)
{
    theRow = r;
    theCol = c;
}
public int getRow()
{ return theRow; }
public int getCol()
{ return theCol; }
}
```

The following `GridPath` class contains the 2D array and methods to use to determine a path through the array. You will write two methods of the `GridPath` class.

```
public class GridPath
{
/** Initialized in the constructor with distinct values that never change */
private int[][] grid;
/**
 * Returns the Location representing a neighbor of the grid element at row and
 col,
 * as described in part (a)
 * Preconditions: row is a valid row index and col is a valid column index in
 grid.
 * row and col do not specify the element in the last row and last column of
 grid.
 */
public Location getNextLoc(int row, int col)
{ /* to be implemented in part (a) */ }
/**
 * Computes and returns the sum of all values on a path through grid, as
 described in
 * part (b)
 * Preconditions: row is a valid row index and col is a valid column index in
 grid.
 * row and col do not specify the element in the last row and last column of
 grid.
 */
public int sumPath(int row, int col)
{ /* to be implemented in part (b) */ }
// There may be instance variables, constructors, and methods that are not
shown.
```

```
}
```

Write the `getNextLoc` method, which returns a `Location` object that represents the smaller of two neighbors of the `grid` element at `row` and `col`, according to the following rules.

The two neighbors that are considered are the element below the given element and the element to the right of the given element, if they exist.

If both neighbors exist, the `Location` of the neighbor with the smaller value is returned. Two neighbors will always have different values.

If only one neighbor exists, the `Location` of the existing neighbor is returned.

For example, assume that `grid` contains the following values.

	0	1	2	3	4
0	12	3	4	13	5
1	11	21	2	14	16
2	7	8	9	15	0
3	10	17	20	19	1
4	18	22	30	25	6

The following table shows some sample calls to `getNextLoc`.

Method Call	Explanation
`getNextLoc(0, 0)`	Returns the neighbor to the right (the `Location` representing the element at row 0 and column 1), since `3 < 11`
`getNextLoc(1, 3)`	Returns the neighbor below (the `Location` representing the element at row 2 and column 3), since `15 < 16`
`getNextLoc(2, 4)`	Returns the neighbor below (the `Location` representing the element at row 3 and column 4), since the given element has no neighbor to the right
`getNextLoc(4, 3)`	Returns the neighbor to the right (the `Location` representing the element at row 4 and column 4), since the given element has no neighbor below

In the example, the `getNextLoc` method will never be called with row `4` and column `4`, as those values would violate the precondition of the method.

Complete the `getNextLoc` method.

```
/**
* Returns the Location representing a neighbor of the grid element at row and col,
* as described in part (a)
* Preconditions: row is a valid row index and col is a valid column index in grid.
* row and col do not specify the element in the last row and last column of grid.
*/
public Location getNextLoc(int row, int col)
```

FINAL Unit4 FRQ - Topic order

Class information for this question

```
public class Location
private int theRow
private int theCol
public Location(int r, int c)
public int getRow()
public int getCol()
public class GridPath
private int[][] grid
public Location getNextLoc(int row, int col)
public int sumPath(int row, int col)
```

Write the `sumPath` method, which returns the sum of all values on a path in `grid`. The path begins with the element at `row` and `col` and is determined by successive calls to `getNextLoc`. The path ends when the element in the last row and the last column of `grid` is reached.

For example, consider the following contents of `grid`. The shaded elements of `grid` represent the values on the path that results from the method call `sumPath(1, 1)`. The method call returns `19` because 3 + 2 + 9 + 4 + 0 + 1 = 19.

	0	1	2	3	4
0	12	30	40	25	5
1	11	3	22	15	43
2	7	2	9	4	0
3	8	33	18	6	1

Write the `sumPath` method. Assume `getNextLoc` works as intended, regardless of what you wrote in part (a). You must use `getNextLoc` appropriately in order to receive full credit.

```
/**
* Computes and returns the sum of all values on a path through grid, as
described in
* part (b)
* Preconditions: row is a valid row index and col is a valid column index in
grid.
* row and col do not specify the element in the last row and last column of
grid.
*/
public int sumPath(int row, int col)
```

Class information for this question

```
public class Location
private int theRow
private int theCol
public Location(int r, int c)
public int getRow()
public int getCol()
public class GridPath
private int[][] grid
```

```
public Location getNextLoc(int row, int col)
public int sumPath(int row, int col)
```

19. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

The `ExperimentalFarm` class represents crops grown on an experimental farm. An experimental farm is a rectangular tract of land that is divided into a grid of equal-sized plots. Each plot in the grid contains one type of crop. The crop yield of each plot is measured in bushels per acre.

A farm plot is represented by the `Plot` class. A partial definition of the `Plot` class is shown below.

```
public class Plot
{
private String cropType;
private int cropYield;

public Plot(String crop, int yield)
{
    /* implementation not shown */
}

public String getCropType()
{
    return cropType;
}

public int getCropYield()
{
    return cropYield;
}
}
```

The grid of equal-sized plots is represented by a two-dimensional array of `Plot` objects named `farmPlots`, declared in the `ExperimentalFarm` class. A partial definition of the `ExperimentalFarm` class is shown below.

```
public class ExperimentalFarm
{
private Plot[][] farmPlots;
```

FINAL Unit4 FRQ - Topic order

```
public ExperimentalFarm(Plot[][] p)
{
    /* implementation not shown */
}

/** Returns the plot with the highest yield for a given crop type, as
described in part (a). */
public Plot getHighestYield(String c)
{
    /* to be implemented in part (a) */
}

/** Returns true if all plots in a given column in the two-dimensional array
farmPlots
 * contain the same type of crop, or false otherwise, as described in part (b).
 */
public boolean sameCrop(int col)
{
    /* to be implemented in part (b) */
}
}
```

(a) Write the `getHighestYield` method, which returns the `Plot` object with the highest yield among the plots in `farmPlots` with the crop type specified by the parameter `c`. If more than one plot has the highest yield, any of these plots may be returned. If no plot exists containing the specified type of crop, the method returns `null`.

Assume that the `ExperimentalFarm` object `f` has been created such that its `farmPlots` array contains the following `cropType` and `cropYield` values.

	0	1	2
0	"corn" 20	"corn" 30	"peas" 10
1	"peas" 30	"corn" 40	"corn" 62
2	"wheat" 10	"corn" 50	"rice" 30
3	"corn" 55	"corn" 30	"peas" 30

The following are some examples of the behavior of the `getHighestYield` method.

Method Call	Return Value

FINAL Unit4 FRQ - Topic order

`f.getHighestYield("corn")`	`farmPlots[1][2]`
`f.getHighestYield("peas")`	`farmPlots[1][0]` or `farmPlots[3][2]`
`f.getHighestYield("bananas")`	`null`

Write the `getHighestYield` method below.

```
/** Returns the plot with the highest yield for a given crop type, as
described in part (a). */
public Plot getHighestYield(String c)
```

(b) Write the `sameCrop` method, which returns `true` if all the plots in a given column of `farmPlots` grow the same crop and returns `false` otherwise.

Assume that the `ExperimentalFarm` object `f` has been created such that its `farmPlots` array contains the following `cropType` and `cropYield` values.

	0	1	2
0	"corn" 20	"corn" 30	"peas" 10
1	"peas" 30	"corn" 40	"corn" 62
2	"wheat" 10	"corn" 50	"rice" 30
3	"corn" 55	"corn" 30	"peas" 30

The following are two examples of the behavior of `sameCrop`.

The method call `f.sameCrop(0)` returns `false` because the values of `cropType` for the elements of column `0` (`"corn"`, `"peas"`, `"wheat"`, and `"corn"`) are not all the same.
The method call `f.sameCrop(1)` returns `true` because the values of `cropType` for all elements of column `1` are the same (`"corn"`).

Write the `sameCrop` method below.

```
/** Returns true if all plots in a given column in the two-dimensional array
farmPlots
* contain the same type of crop, or false otherwise, as described in part (b).
*/
public boolean sameCrop(int col)
```